Binary Search Notes

1. Binary Search Implementation
    Simple Binary Search
    Invariant Binary Search
    Theory of invariance
2. BS Questions
    2.1 LC704: Binary Search
    2.2 Implement Lower Bound
    2.3 Implement Upper Bound
    2.4 LC35: Search Insert Position
    2.5 Floor and Ceil in Sorted Array
    2.6 LC34: First and Last Occurrence
    2.7 Count the occurrence in sorted array
    2.8 LC33: Search in Rotated Sorted Array
    2.9 LC81: **Search in Rotated Sorted Array II**
    2.10 LC153: Minimum in Rotated Sorted Array
    2.11 Find out how many times the array is rotated
    2.12 LC540: Single Element in a sorted array
    2.13 LC162: Find Peak Element
3. Binary Search on Answer
    Monotonous Insight
    Binary Search Minimax Problems
4. BSA Questions
    4.1 **Square root of number upto given precision**
    4.2 Find nth root of a number
    4.3 LC875: Koko eating bananas
    4.4 LC1482: Minimum days to make M bouquets
    4.5 LC**1283: Find the Smallest Divisor Given a Threshold**
    4.6 LC1011: Capacity to ship packages within D days
    4.7 LC1539: Kth missing positive number
    4.8 Aggressive Cows
    4.9 Book Allocation Problem
    4.10 LC410:Split Array Largest Sum
    4.11 Painter's Partition Problem
    4.12 Minimize Max Distance to Gas Station
    4.13 LC4: Median of two sorted arrays
    4.14 Kth Element of two sorted arrays
5. Binary Search on 2D Arrays
    5.1 Find row with maximum 1's
    5.2 LC74:Search in sorted 2D matrix
    5.3 LC240: Search in sorted matrix II
    5.4 LC1901: Find Peak Element II
    5.5 Matrix Median

1. Binary Search Implementation

Simple Binary Search

```
//basic binary search

int binarySearch(vector<int> & arr, int target){
    int low = 0, high = arr.size()-1;
    while(low <= high){
        int mid = low + (high - low)/2;
        if(arr[mid] == target){
            return mid;
        }else if(arr[mid] < target){
            low = mid + 1;
        }else{
            high = mid - 1;
        }
    }
    return -1;
}
//Time Complexity: O(log(n)) | Space Complexity: O(1)
```

Invariant Binary Search

```
//invariant BS(Codeforces)
int l = - 1; // virtual element: INT_MIN
int r = n;   // virtual element: INT_MAX
while(r > l + 1){
    int m = l + (r - l)/2;
    if(a[m] < x){
        l = m;
    }else{
```

```
        r = m;
    }
}
if(r < n && a[r] == x) {
    cout << r;
}else{
    cout << -1;
}
```

Theory of invariance

```
.....0 1 1 2 .....
     l r         <- at last iteration
target(x) = 1
[invariant]:[  l < r  ]
            [ aₗ  <  x ]
            [ aᵣ >= x ]
Equality at r will give first occurence of 1


.....0 1 1 2 .....
         l r    <- at last iteration
[invariant]:[  l < r   ]
            [ aₗ  <=  x ]
            [ aᵣ > x   ]
Equality at l will give last occurence of 1


invariant:
            [ aₗ  <  x ]
            [ aᵣ >= x ]
At last, convergence will be
           l    r
           aᵢ | aᵢ₊₁
```

2. BS Questions

2.1 LC704: Binary Search

```
//https://leetcode.com/problems/binary-search/description/
// Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search
target in nums. If target exists, then return its index. Otherwise, return -1.
// You must write an algorithm with O(log n) runtime complexity.
func search(nums []int, target int) int {
    l := -1
    r := len(nums)
    for r > l + 1 {
        mid := l + (r-l)/2
        if nums[mid] < target {
            l = mid;
        }else{
            r = mid;
        }
    }
    if r < len(nums) && nums[r] == target {
        return r;
    }
    return -1;
}
```

2.2 Implement Lower Bound

```
// https://www.naukri.com/code360/problems/lower-bound_81653
// You are given an array 'arr' sorted in non-decreasing order and a number 'x'. You must return the index of the lower b
ound of 'x'.

// Lower Bound : idx such that arr[idx] is not less that x (<=)
int lowerBound(vector<int> arr, int n, int x) {
        int l = -1;
        int r = n;
        while(r > l + 1){
            int mid = l + (r - l)/2;
```

```
            if(arr[mid] < x){
                l = mid;
            }else{
                r = mid;
            }
        }
        return r;
}
```

2.3 Implement Upper Bound

```
//https://www.naukri.com/code360/problems/implement-upper-bound_8165383?leftPanelTabValue=PROBLEM
// You are given a sorted array `arr` containing `n` integers and an integer `x`.
// Implement the `upper bound` function to find the index of the upper bound of 'x' in the array.

// Upper Bound: in a sorted array is the index of the first value that is greater than a given value.
int upperBound(vector<int> &arr, int x, int n){
        int l = -1;
        int r = n;
        while(r > l + 1){
            int mid = l + (r - l)/2;
            if(arr[mid] <= x){
                l = mid;
            }else{
                r = mid;
            }
        }
        return r;
}
```

2.4 LC35: Search Insert Position

```
//https://leetcode.com/problems/search-insert-position/
// Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return
the index where it would be if it were inserted in order.

func searchInsert(nums []int, target int) int {
    l, r := -1, len(nums)
    for r > l+1 {
        mid := l + (r-l)/2
        if nums[mid] < target {
            l = mid
        } else {
            r = mid
        }
    }
    return r
}
```

2.5 Floor and Ceil in Sorted Array

```
//https://www.naukri.com/code360/problems/ceiling-in-a-sorted-array_1825401
//You're given a sorted array 'a' of 'n' integers and an integer 'x'.
// Find the floor and ceiling of 'x' in 'a[0..n-1]'.
// Note:
// Floor of 'x' is the largest element in the array which is smaller than or equal to 'x'.
// Ceiling of 'x' is the smallest element in the array greater than or equal to 'x'.

pair<int, int> getFloorAndCeil(vector<int> &a, int n, int x) {
    int l = -1;
    int r = n;
    while(r > l + 1){
        int mid = l + (r - l)/2;
        if(a[mid] < x){
            l = mid;
        }else{
            r = mid;
        }
    }
    if(r < n && a[r] == x){
        return {a[r], a[r]};
    }
    if( l == -1){
        return {-1, a[0]};
    }
    if( r == n){
```

```
        return {a[l], -1};
    }
    return {a[l], a[r]};
}
```

2.6 LC34: First and Last Occurrence

```
//https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/
// Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given targe
t value.
// If target is not found in the array, return [-1, -1].

class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        auto first = lower_bound(nums.begin(), nums.end(), target);
        if (first == nums.end() || *first != target) return {-1, -1};
        auto last = upper_bound(nums.begin(), nums.end(), target) - 1;
        return {(int)(first - nums.begin()), (int)(last - nums.begin())};
    }
};
```

2.7 Count the occurrence in sorted array

```
// https://www.naukri.com/code360/problems/occurrence-of-x-in-a-sorted-array_630456
// You have been given a sorted array/list of integers 'arr' of size 'n' and an integer 'x'.
// Find the total number of occurrences of 'x' in the array/list.

// Pro_solution:
#include <algorithm>
int count(vector<int>& arr, int n, int x) {
    auto lo = lower_bound(arr.begin(), arr.end(), x);
    if (lo == arr.end() || *lo != x) return 0;
    auto hi = upper_bound(arr.begin(), arr.end(), x);
    return hi - lo;
}

// int firstOccurence(vector<int>& arr, int n, int x){
//     int l = -1;
//     int r = n;
//     while(r > l + 1){
//         int mid = l + (r - l)/2;
//         if(arr[mid] < x){
//             l = mid;
//         }else{
//             r = mid;
//         }
//     }
//     if(r < n && arr[r] == x){
//         return r;
//     }
//     return -1;
// }

// int lastOccurence(vector<int>& arr, int n, int x){
//     int l = -1;
//     int r = n;
//     while(r > l + 1){
//         int mid = l + (r - l)/2;
//         if(arr[mid] <= x){
//             l = mid;
//         }else{
//             r = mid;
//         }
//     }
//     if(l != -1 && arr[l] == x){
//         return l;
//     }
//     return -1;
// }

// int count(vector<int>& arr, int n, int x) {
//     int l = lastOccurence(arr, n, x);
//     int r = firstOccurence(arr, n, x);
//     if(l == -1){
//         return 0;
```

```
//     }
//     return l - r + 1;
// }
```

2.8 LC33: Search in Rotated Sorted Array

```
// https://leetcode.com/problems/search-in-rotated-sorted-array/
//There is an integer array nums sorted in ascending order (with distinct values).
//Prior to being passed to your function, nums is possibly left rotated at an unknown index k (1 <= k < nums.length) such
that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For examp
le, [0,1,2,4,5,6,7] might be left rotated by 3 indices and become [4,5,6,7,0,1,2].
//Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or
-1 if it is not in nums.

//1. simple binary search(l<=r)
//2. At every iteration either left half or right half will be sorted, if target lies in that part, double down to it.
//3. for simplicity, use equality everywhere (<= & >=)

package main
func search(nums []int, target int) int {
    l := 0
    r := len(nums) - 1
    for l<=r {
        mid := (l + r)/2
        if(nums[mid]==target){
            return mid;
        }
        if nums[mid] >= nums[l] {
            //left part is sorted
            if nums[mid] >= target && nums[l] <= target {
                r = mid - 1
            }else{
                l = mid + 1
            }
        }else{
            //right part is sorted
            if nums[mid] <= target && nums[r] >= target{
                l = mid + 1
            }else{
                r = mid - 1
            }
        }
    }
    return -1;
}
```

2.9 LC81: Search in Rotated Sorted Array II

```
//https://leetcode.com/problems/search-in-rotated-sorted-array-ii/
//There is an integer array nums sorted in non-decreasing order (not necessarily with distinct values).
//Before being passed to your function, nums is rotated at an unknown pivot index k (0 <= k < nums.length) such that the
resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,
2,4,4,4,5,6,6,7] might be rotated at pivot index 5 and become [4,5,6,6,7,0,1,2,4,4].
//Given the array nums after the rotation and an integer target, return true if target is in nums, or false if it is not
in nums.

// [3 1 2 3 3 3 3]
//  l     m     h
// arr[l] = arr[m] = arr[h] => can't do comparions of arr[l] <= arr[m] like normally do
// so to trim down the search space, lets observe what makes current code to fail.

func search(nums []int, target int) bool {
    l := 0
    r := len(nums) - 1

    for l <= r {
        mid := (l + r) / 2

        if nums[mid] == target {
            return true
        }

        //add following update to trim down the search space
        if nums[l] == nums[mid] && nums[r] == nums[mid] {
            l++
            r--
```

```
            continue
        }

        if nums[l] <= nums[mid] {
            if nums[l] <= target && target <= nums[mid] {
                r = mid - 1
            } else {
                l = mid + 1
            }
        } else {
            if nums[mid] <= target && target <= nums[r] {
                l = mid + 1
            } else {
                r = mid - 1
            }
        }
    }

    return false
}
```

2.10 LC153: Minimum in Rotated Sorted Array

```
//https://leetcode.com/problems/find-minimum-in-rotated-sorted-array/
// Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array nums =
[0,1,2,4,5,6,7] might become:
// [4,5,6,7,0,1,2] if it was rotated 4 times.
// [0,1,2,4,5,6,7] if it was rotated 7 times.
// Notice that rotating an array [a[0], a[1], a[2], ..., a[n-1]] 1 time results in the array [a[n-1], a[0], a[1], a[2],
..., a[n-2]].
// Given the sorted rotated array nums of unique elements, return the minimum element of this array.


// 1. Minimum lies in unsorted part
// 2. Edge Case: If whole part is sorted (nums[l] <= nums[r]) return nums[l]
// 3. prev := (mid - 1 + n)%n && next := (mid + 1)%n

func findMin(nums []int) int {
    l := 0
    n := len(nums)
    r := n - 1

    for l <= r {
        mid := l + (r - l)/2
        prev := (mid - 1 + n) % n
        next := (mid + 1) % n
        if(nums[mid] < nums[next] && nums[mid] < nums[prev]){
            return nums[mid];
        }
        if nums[l] <= nums[r]{
            return nums[l];
        }
        if nums[l] <= nums[mid]{
            l = mid + 1
        }else{
            r = mid - 1;
        }
    }
    return -1;
}
```

2.11 Find out how many times the array is rotated

Same logic as previous

2.12 LC540: Single Element in a sorted array

```
//https://leetcode.com/problems/single-element-in-a-sorted-array/description/
// You are given a sorted array consisting of only integers where every element appears exactly twice, except for one ele
ment which appears exactly once.
// Return the single element that appears only once.

// 0. Its an odd even game
// 1. Use prev & next
// 2. Edge Case: if array size is 1
func singleNonDuplicate(nums []int) int {
    l := 0
```

```go
    n := len(nums)
    r := n - 1
    if n == 1 {
        return nums[0]
    }
    for l <= r {
        mid := l + (r - l)/2
        prev := (mid - 1 + n )%n
        next := (mid + 1) %n
        if(nums[mid]!= nums[prev] && nums[mid]!= nums[next]){
            return nums[mid];
        }
        if (mid%2 == 1 && nums[prev] != nums[mid]) || (mid%2==0 && nums[prev] == nums[mid]){
            r = mid - 1
        }else{
            l = mid + 1
        }
    }
    return -1
}
```

2.13 LC162: Find Peak Element

```go
// https://leetcode.com/problems/find-peak-element/
// A peak element is an element that is strictly greater than its neighbors.
// Given a 0-indexed integer array nums, find a peak element, and return its index. If the array contains multiple peaks,
return the index to any of the peaks.
// You may imagine that nums[-1] = nums[n] = -∞. In other words, an element is always considered to be strictly greater t
han a neighbor that is outside the array.

// Why this works: if nums[mid] < nums[mid+1], the right side is ascending so a peak must exist there. Otherwise, mid its
elf could be the peak or the peak is to the left — so shrink r to mid (not mid-1, since mid is still a candidate). Loop e
xits when l == r, which is the peak.
func findPeakElement(nums []int) int {
    l, r := 0, len(nums)-1
    for l < r {
        mid := (l + r) / 2
        if nums[mid] < nums[mid+1] {
            l = mid + 1
        } else {
            r = mid
        }
    }
    return l
}

// Edge Case: Array size one, first & last element peak
// Assume there is only one peak. Peak lies in non ascending part (nums[mid] > nums[mid - 1] {
//             l = mid + 1)
// func findPeakElement(nums []int) int {
//     if len(nums) == 1 {
//         return 0
//     }
//     if nums[0] > nums[1] {
//         return 0
//     }
//     if nums[len(nums)-1] > nums[len(nums)-2] {
//         return len(nums)-1
//     }
//     l := 1
//     r := len(nums)-2
//     for l<=r {
//         mid := (l+r)/2
//         if nums[mid] > nums[mid - 1] && nums[mid] > nums[mid + 1]{
//             return mid
//         } else if nums[mid] > nums[mid - 1] {
//             l = mid + 1
//         }else {
//             r = mid - 1
//         }
//     }
//     return -1
// }
```

3. Binary Search on Answer

Monotonous Insight

If `x` is good, then `x + 1` is also good.
Monotonic property, once true, stays true.

$$f(x) = \begin{cases} 1 & \text{if } x \geq \text{goodNumber} \\ 0 & \text{otherwise} \end{cases}$$

$$\underbrace{0 \quad 0 \quad 0 \quad 0 \quad 0}_{\text{bad (f = 0)}} \Big| \underbrace{1 \quad 1 \quad 1 \quad 1 \quad 1}_{\text{good (f = 1)}}$$

$$\text{Answer} = \min\{x \mid f(x) = 1\} \quad \leftarrow \text{inflection point}$$

Create a function to check whether `x` is good or bad.

$$\begin{array}{ccccccccccc} \cdots & 0 & 0 & 0 & 0 & \Big| & 1 & 1 & 1 & 1 & \cdots \\ & & & & \uparrow & & \uparrow & & & & \longrightarrow x \\ & & & & l & & r & & & & \end{array}$$

If `f(m) = 1` , *then m is good number,* `r = m`, *otherwise* `f(m) = 0` , *hence m is bad so* `l = m` , *return* `r` *at last.*

$$f(l) = 0 \quad \text{always}, \qquad f(r) = 1 \quad \text{always}$$
$$\text{Loop terminates when } r = l + 1 \implies \text{answer} = r$$

```
// Example: first index where a[i] >= x  (lower bound)

auto feasible = [&](int m) -> bool {
    return a[m] >= x;
};

int l = -1;    // virtual: f(l) = 0 always  (sentinel below range)
int r = n;     // virtual: f(r) = 1 always  (sentinel above range)

while (r > l + 1) {
    int m = l + (r - l) / 2;
    if (feasible(m)) {
        r = m;   // m could be answer, pull r in
    } else {
        l = m;   // m is bad, push l out
    }
}

// r = smallest index where f(r) = 1
// l = largest index where f(l) = 0
return r;
```

Binary Search Minimax Problems

`min(max(…))` or `max(min(…))`

Example:
Let us have a number line on which people live. For each person `i` we know the coordinate of house `xᵢ`. We want to collect them at one point. How long can people get together at this point? Depends on how fast people walk, `i-th` person has `vᵢ` speed. So each person will take `| xᵢ - x | / vᵢ` time to reach point x. Time for which all people will come to this point - the maximum of these values. We want to minimize this time. i.e. we need to find `x` for following expression.

$$\min_x \left(\max_i \left(\frac{x_i - x}{v_i} \right) \right)$$

$$\max_i \left(\frac{x_i - x}{v_i} \right) \leq t$$

this is monotone. If `t` fits the condition for any `y > 0` . `t + y` also fits.

$$\frac{x_i - x}{v_i} \leq t$$

$$z_i \leq t \quad \text{where } z_i = \frac{x_i - x}{v_i}$$

if `max(zᵢ)` satisfies this inequality, then all `zᵢ` will satisfy this inequality.

$$|x_i - x| \leq t v_i$$

$$x \in [x_i - t v_i,\ x_i + t v_i]$$

Find all segments. For a given x, if all segments $[x_i - t v_i,\ x_i + t v_i]$ intersection is non-empty, then, `x` exists & hence people can meet at coordinate `x` . We just need to check whether **problem is `x` solvable or not.**

```
bool solvable(double X, int n, vector<int>&xi, vector<int>&vi){
    double L = -1e18;
    double R = 1e18;
    for(int i = 0; i<n;i++){
        double lB = xi[i] - X * vi[i];
        double rB = xi[i] + X * vi[i];
        if( rB >= lB ) {
            L = max(L, lB);
            R = min(R, rB);
        }
    }
    double dist = R - L;
    return dist >= 0;
}

double minMaxBin(int n, vector<int>& xi, vector<int>& vi{
    double l = 0.0;
    double r = 1.0;
    while(!solvable(r, n, xi, vi)){
        r *= 2.0;
    }
    for(int i = 0; i < 100; i++){
        double mid = (l+r)/2;
        if(solvable(mid, n, xi, vi)){
            r = mid;
        }else{
            l = mid;
        }
    }
    return r;
}
```

4. BSA Questions

4.1 Square root of number upto given precision

```
//https://www.geeksforgeeks.org/problems/square-root-of-number-upto-given-precision/1
// Given a positive number n and an integer p representing the desired precision, your task is to find the square root of
n, accurate to p decimal places. The solution should avoid using built-in square root functions.
// Note: Precision control is required to ensure the output is correctly rounded or truncated at p digits after the decim
al.

//Learn the trick to truncate
#include <cmath>

class Solution {
  public:
    double squareRoot(int n, int p) {
        double l = 1.0;
        double r = 1.0;

        while (r * r < n) {
            r = r * 2.0;
        }

        for (int i = 0; i < 100; i++) {
            double mid = l + (r - l) / 2.0;
            if (mid * mid < n) {
                l = mid;
            } else {
                r = mid;
            }
        }

        double factor = pow(10, p);
        double eps = 1e-12;
        return floor((r + eps) * factor) / factor;
    }
};
```

4.2 Find nth root of a number

```
//https://www.naukri.com/code360/problems/nth-root-of-m_1062679
// Problem statement
// You are given two positive integers 'n' and 'm'. You have to return the 'nth' root of 'm', i.e. 'm(1/n)'. If the 'nth
```

```
root is not an integer, return -1.
// Note:
// 'nth' root of an integer 'm' is a number, which, when raised to the power 'n', gives 'm' as a result.

// 1. long long
// 2. > == <
int power(int x, int n, int m){
  long long pow = 1L;
  while(n > 0){
    pow *= x;
    n--;
    if(pow > m){
      return 2;
    }
  }
  if(pow == m ) return 1;
  if(pow > m ) return 2;
  return 0;
}

int NthRoot(int n, int m) {
  int l = 0;
  int r = m;
  while(r > l + 1){
    int mid = l + (r-l)/2;
    if(power(mid, n, m) == 0){
      l = mid;
    }else{
      r = mid;
    }
  }
  if(power(r, n, m)==1){
    return r;
  }
  return -1;
}
```

4.3 LC875: Koko eating bananas

```
// https://www.geeksforgeeks.org/problems/minimize-max-distance-to-gas-station/1
// Minimize Max Distance to Gas Station
// Difficulty: HardAccuracy: 38.36%Submissions: 122K+Points: 8Average Time: 40m
// We have a horizontal number line. On that number line, we have gas stations at positions stations[0], stations[1],
..., stations[n-1]. Now, we add k more gas stations so that d, the maximum distance between adjacent gas stations, is min
imized. We have to find the smallest possible value of d. Find the answer exactly to 6 decimal places.
// Note: stations is in a strictly increasing order.
// Examples:
// Input: stations[] = [1, 2, 3, 4, 5], k = 2
// Output: 1.00
// Explanation: Since all gaps are already equal (1 unit each), adding extra stations in between does not reduce the maxi
mum distance.

class Solution {
  public:
    bool solvable(double x, vector<int> &stations, int K){
        int z = K;
        for(int i = 1;i< stations.size();i++){
            int dist = stations[i] - stations[i-1];
            int canBePut = (int)ceil((double)dist / x) - 1;
            z -= canBePut;
        }
        if(z<0){
            return false;
        }
        return true;
    }

    double minMaxDist(vector<int> &stations, int K) {
        int n = stations.size();
        double l = 0.0;
        double r = 1.0;
        while(!solvable(r, stations, K)){
            r *= 2.0;
        }
        while(r - l > 0.000001){
            double mid = (r + l)/2.0;
            if(!solvable(mid, stations, K)){
```

```
                l = mid;
            }else{
                r = mid;
            }
        }
        return r;
    }
};
```

```
//https://leetcode.com/problems/koko-eating-bananas/
//Koko loves to eat bananas. There are n piles of bananas, the ith pile has piles[i] bananas. The guards have gone and wi
ll come back in h hours.
//Koko can decide her bananas-per-hour eating speed of k. Each hour, she chooses some pile of bananas and eats k bananas
from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas durin
g this hour.
//Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.
//Return the minimum integer k such that she can eat all the bananas within h hours.
//Example 1:
//Input: piles = [3,6,7,11], h = 8
//Output: 4

//Easy
func monotone(piles []int, h int, k int) bool {
    ans := 0
    for _, v := range piles {
        if v < k {
            ans += 1
        }else if v%k == 0 {
            ans += v/k
        } else {
            ans += 1 + v/k
        }
    }
    return h >= ans
}

func minEatingSpeed(piles []int, h int) int {
    // monotone funtion
    l := 0
    r := 100000000000
    for r > l + 1 {
        k := (l + (r-l)/2)
        if monotone(piles, h, k) {
            r = k;
        }else{
            l = k;
        }
    }
    return r;
}
```

4.4 LC1482: Minimum days to make M bouquets

```
// https://leetcode.com/problems/minimum-number-of-days-to-make-m-bouquets/description/
// You are given an integer array bloomDay, an integer m and an integer k.
// You want to make m bouquets. To make a bouquet, you need to use k adjacent flowers from the garden.
// The garden consists of n flowers, the ith flower will bloom in the bloomDay[i] and then can be used in exactly one bou
quet.
// Return the minimum number of days you need to wait to be able to make m bouquets from the garden. If it is impossible
to make m bouquets return -1.
// Input: bloomDay = [7,7,7,7,12,7,7], m = 2, k = 3
// Output: 12
// Explanation: We need 2 bouquets each should have 3 flowers.
// Here is the garden after the 7 and 12 days:
// After day 7: [x, x, x, x, _, x, x]
// We can make one bouquet of the first three flowers that bloomed. We cannot make another bouquet from the last three fl
owers that bloomed because they are not adjacent.
// After day 12: [x, x, x, x, x, x, x]
// It is obvious that we can make two bouquets in different ways.

func minDays(bloomDay []int, m, k int) int {
    if m*k > len(bloomDay) {
        return -1
    }

    lo, hi := slices.Min(bloomDay)-1, slices.Max(bloomDay)
```

```go
    for hi > lo+1 {
        mid := lo + (hi-lo)/2
        if canMakeBouquets(bloomDay, m, k, mid) {
            hi = mid
        } else {
            lo = mid
        }
    }
    return hi
}

// canMakeBouquets checks if m bouquets of k adjacent bloomed flowers are possible by `day`.
func canMakeBouquets(bloomDay []int, m, k, day int) bool {
    bouquets, streak := 0, 0
    for _, d := range bloomDay {
        if d <= day {
            streak++
        } else {
            streak = 0
        }
        if streak == k {
            bouquets++
            streak = 0
        }
    }
    return bouquets >= m
}
```

4.5 LC1283: Find the Smallest Divisor Given a Threshold

```go
//https://leetcode.com/problems/find-the-smallest-divisor-given-a-threshold/description/
// Given an array of integers nums and an integer threshold, we will choose a positive integer divisor, divide all the ar
ray by it, and sum the division's result. Find the smallest divisor such that the result mentioned above is less than or
equal to threshold.
// Each result of the division is rounded to the nearest integer greater than or equal to that element. (For example: 7/3
= 3 and 10/2 = 5).
// The test cases are generated so that there will be an answer.
// Example 1:
// Input: nums = [1,2,5,9], threshold = 6
// Output: 5
// Explanation: We can get a sum to 17 (1+2+5+9) if the divisor is 1.
// If the divisor is 4 we can get a sum of 7 (1+1+2+3) and if the divisor is 5 the sum will be 5 (1+1+1+2).
func smallestDivisor(nums []int, threshold int) int {
    lo, hi := 0, slices.Max(nums) // lo=0 is invalid (never evaluated), hi is always valid
    for hi > lo+1 {
        mid := lo + (hi-lo)/2
        if divSum(nums, mid) <= threshold {
            hi = mid
        } else {
            lo = mid
        }
    }
    return hi
}

func divSum(nums []int, divisor int) int {
    total := 0
    for _, n := range nums {
        total += (n + divisor - 1) / divisor
    }
    return total
}
```

4.6 LC1011: Capacity to ship packages within D days

```go
// https://leetcode.com/problems/capacity-to-ship-packages-within-d-days/
// A conveyor belt has packages that must be shipped from one port to another within days days.
// The ith package on the conveyor belt has a weight of weights[i]. Each day, we load the ship with packages on the conve
yor belt (in the order given by weights). We may not load more weight than the maximum weight capacity of the ship.
// Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped w
ithin days days.
// Example 1:
// Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5
// Output: 15
// Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:
// 1st day: 1, 2, 3, 4, 5
```

```go
// 2nd day: 6, 7
// 3rd day: 8
// 4th day: 9
// 5th day: 10
// Note that the cargo must be shipped in the order given, so using a ship of capacity 14 and splitting the packages into parts like (2, 3, 4, 5), (1, 6, 7), (8), (9), (10) is not allowed.
func shipWithinDays(weights []int, days int) int {
    lo, hi := slices.Max(weights)-1, sum(weights) // lo fails (can't fit max), hi always works
    for hi > lo+1 {
        mid := lo + (hi-lo)/2
        if canShip(weights, days, mid) {
            hi = mid
        } else {
            lo = mid
        }
    }
    return hi
}

func canShip(weights []int, days, capacity int) bool {
    daysNeeded, load := 1, 0
    for _, w := range weights {
        if load+w > capacity {
            daysNeeded++
            load = 0
        }
        load += w
    }
    return daysNeeded <= days
}

func sum(nums []int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}
```

4.7 LC1539: Kth missing positive number

```cpp
//https://leetcode.com/problems/kth-missing-positive-number/
// Given an array arr of positive integers sorted in a strictly increasing order, and an integer k.
// Return the kth positive integer that is missing from this array.
// Example 1:
// Input: arr = [2,3,4,7,11], k = 5
// Output: 9
// Explanation: The missing positive integers are [1,5,6,8,9,10,12,13,...]. The 5th missing positive integer is 9.


class Solution {
public:
    int findKthPositive(vector<int>& arr, int k) {
        int l = 0;
        int r = arr.size()-1;
        while(l<=r){
            int mid = (l + r)/2;
            int missing = arr[mid] - mid - 1;
            if(missing < k){
                l = mid + 1;
            }else{
                r = mid - 1;
            }
        }
        return l + k;
    }
};
```

4.8 Aggressive Cows

```cpp
// https://www.naukri.com/code360/problems/aggressive-cows_1082559?leftPanelTabValue=PROBLEM
// Problem statement
// You are given an array 'arr' consisting of 'n' integers which denote the position of a stall.
// You are also given an integer 'k' which denotes the number of aggressive cows.
// You are given the task of assigning stalls to 'k' cows such that the minimum distance between any two of them is the m
```

```
aximum possible.
// Print the maximum possible minimum distance.
// Example:
// Input: 'n' = 3, 'k' = 2 and 'arr' = {1, 2, 3}
// Output: 2
// Explanation: The maximum possible minimum distance will be 2 when 2 cows are placed at positions {1, 3}. Here distance
between cows is 2.

#include <algorithm>
bool solvable(int d, vector<int> &stalls, int k){
    //k cows, d distance in each cow & stalls = stalls
    int z = k - 1;
    int location = stalls[0];
    for(int i = 1; i < stalls.size(); i++){
        int loc = stalls[i];
        if(loc - location >=d){
            z--;
            location = loc;
        }else{
            continue;
        }
        if(z==0){
            return true;
        }
    }
    return false;
}

int aggressiveCows(vector<int> &stalls, int k)
{
    sort(stalls.begin(), stalls.end());
    //d solvable problem
    int l = 0;
    int r = 1e9 + 1;
    while( r > l + 1){
        int mid = l + (r - l)/2;
        if(solvable(mid, stalls, k)){
            l = mid;
        }else{
            r = mid;
        }
    }
    return l;
}
```

4.9 Book Allocation Problem

```
// https://www.naukri.com/code360/problems/allocate-books_1090540
// Problem statement
// Given an array ?arr? of integer numbers, ?arr[i]? represents the number of pages in the ?i-th? book.
// There are ?m? number of students, and the task is to allocate all the books to the students.
// Allocate books in such a way that:
// 1. Each student gets at least one book.
// 2. Each book should be allocated to only one student.
// 3. Book allocation should be in a contiguous manner.
// You have to allocate the book to ?m? students such that the maximum number of pages assigned to a student is minimum.
// If the allocation of books is not possible, return -1.
// Example:
// Input: ?n? = 4 ?m? = 2
// ?arr? = [12, 34, 67, 90]
// Output: 113
// #include <algorithm>

bool solvable(int x, vector<int>& arr, int m) {
    int pages = 0;
    int students = 1;
    for (int i = 0; i < arr.size(); i++) {
        if (pages + arr[i] <= x) {
            pages += arr[i];
        } else {
            students++;
            pages = arr[i];
        }
    }
    return students <= m;
}
```

```
int findPages(vector<int>& arr, int n, int m) {
    if (m > n || m < 1) return -1;
    int l = 0;
    int r = 1e9 + 1;
    while (r > l + 1) {
        int mid = l + (r - l) / 2;
        if (solvable(mid, arr, m)) {
            r = mid;   // mid works, try smaller
        } else {
            l = mid;   // mid too small
        }
    }
    return r;
}
```

4.10 LC410:Split Array Largest Sum

```
//https://leetcode.com/problems/split-array-largest-sum/description/
// Given an integer array nums and an integer k, split nums into k non-empty subarrays such that the largest sum of any s
ubarray is minimized.
// Return the minimized largest sum of the split.
// A subarray is a contiguous part of the array.
// Example 1:
// Input: nums = [7,2,5,10,8], k = 2
// Output: 18
// Explanation: There are four ways to split nums into two subarrays.
// The best way is to split it into [7,2,5] and [10,8], where the largest sum among the two subarrays is only 18.
func solvable(x int, nums []int, k int) bool {
    z := k - 1
    t := 0
    for _, v := range nums {
        if t+v <= x {
            t += v
        } else {
            z--
            t = v
        }
    }
    return z >= 0
}

func splitArray(nums []int, k int) int {
    maxVal := 0
    for _, v := range nums {
        if v > maxVal {
            maxVal = v
        }
    }

    l := maxVal - 1
    r := 1000000001
    for r > l+1 {
        mid := l + (r-l)/2
        if solvable(mid, nums, k) {
            r = mid
        } else {
            l = mid
        }
    }
    return r
}
```

4.11 Painter's Partition Problem

```
// https://www.naukri.com/code360/problems/painter-s-partition-problem_1089557
// Given an array/list of length ?n?, where the array/list represents the boards and each element of the given array/list
represents the length of each board. Some ?k? numbers of painters are available to paint these boards. Consider that each
unit of a board takes 1 unit of time to paint.
// You are supposed to return the area of the minimum time to get this job done of painting all the ?n? boards under a co
nstraint that any painter will only paint the continuous sections of boards.
// Example :
// Input: arr = [2, 1, 5, 6, 2, 3], k = 2
// Output: 11
// Explanation:
// First painter can paint boards 1 to 3 in 8 units of time and the second painter can paint boards 4-6 in 11 units of ti
me. Thus both painters will paint all the boards in max(8,11) = 11 units of time. It can be shown that all the boards ca
```

```
n't be painted in less than 11 units of time.
bool solvable(int x, vector<int>& arr, int k) {
    int z = k - 1;
    int t = 0;
    for (int v : arr) {
        if (t + v <= x) {
            t += v;
        } else {
            z--;
            t = v;
        }
    }
    return z >= 0;
}

int findLargestMinDistance(vector<int> &boards, int k){
    int maxVal = *max_element(boards.begin(), boards.end());

    int l = maxVal - 1;
    int r = 1000000001;
    while (r > l + 1) {
        int mid = l + (r - l) / 2;
        if (solvable(mid, boards, k)) {
            r = mid;
        } else {
            l = mid;
        }
    }
    return r;
}
```

4.12 Minimize Max Distance to Gas Station

4.13 LC4: Median of two sorted arrays

```
func count(target int, nums []int) int {
    l := -1
    r := len(nums)
    for r > l + 1 {
        mid := l + (r-l)/2
        if nums[mid] <= target {
            l = mid
        }else{
            r = mid
        }
    }
    return l
}

//find xth element in combined sorted array
func element(x int, nums1 []int, nums2 []int) int {
    if len(nums1) == 0 {
        return nums2[x-1]
    }
    if len(nums2) == 0 {
        return nums1[x-1]
    }

    minVal := nums1[0]
    if nums2[0] < minVal {
        minVal = nums2[0]
    }
    l := minVal - 1
    maxVal := nums1[len(nums1)-1]
    if nums2[len(nums2)-1] > maxVal {
        maxVal = nums2[len(nums2)-1]
    }
    r := maxVal + 1
    for r > l+1 {
        cnt := 0
        mid := l + (r-l)/2
        cnt += count(mid, nums1) + 1
        cnt += count(mid, nums2) + 1
        if cnt < x {
            l = mid
        } else {
            r = mid
```

```
        }
    }
    return r
}

func findMedianSortedArrays(nums1 []int, nums2 []int) float64 {
    n := len(nums1)
    m := len(nums2)
    totalSize := n + m
    if totalSize%2 == 1 {
        tofind :=  totalSize/2 + 1
        return float64(element(tofind, nums1, nums2))
    }else{
        tofind := totalSize/2
        tofind2 := 1 + totalSize/2
        e1 := element(tofind, nums1, nums2)
        e2 := element(tofind2, nums1, nums2)
        return (float64(e1) + float64(e2))/2.0
    }
}
```

4.14 Kth Element of two sorted arrays

```
// https://www.naukri.com/code360/problems/k-th-element-of-2-sorted-array_1164159
// Problem statement
// You're given two sorted arrays 'arr1' and 'arr2' of size 'n' and 'm' respectively and an element 'k'.
// Find the element that would be at the 'kth' position of the combined sorted array.
// Position 'k' is given according to 1 - based indexing, but arrays 'arr1' and 'arr2' are using 0 - based indexing.
// For example :
// Input: 'arr1' = [2, 3, 45], 'arr2' = [4, 6, 7, 8] and 'k' = 4
// Output: 6
// Explanation: The merged array will be [2, 3, 4, 6, 7, 8, 45]. The element at position '4' of this array is 6. Hence we
return 6.

int countLE(int target, vector<int>& arr) {
    int l = -1, r = arr.size();
    while (r > l + 1) {
        int mid = l + (r - l) / 2;
        if (arr[mid] <= target) l = mid;
        else r = mid;
    }
    return l;
}

int kthElement(vector<int>& arr1, vector<int>& arr2, int n, int m, int k) {
    int l = min(arr1[0], arr2[0]) - 1;
    int r = max(arr1[n-1], arr2[m-1]) + 1;

    while (r > l + 1) {
        int mid = l + (r - l) / 2;
        int cnt = (countLE(mid, arr1) + 1) + (countLE(mid, arr2) + 1);
        if (cnt < k) l = mid;
        else r = mid;
    }
    return r;
}
```

5. Binary Search on 2D Arrays

5.1 Find row with maximum 1's

```
// https://www.geeksforgeeks.org/problems/row-with-max-1s0023/1
// You are given a 2D binary array arr[][] consisting of only 1s and 0s. Each row of the array is sorted in non-decreasin
g order. Your task is to find and return the index of the first row that contains the maximum number of 1s. If no such ro
w exists, return -1.
// Note:
// The array follows 0-based indexing.
// The number of rows and columns in the array are denoted by n and m respectively.
// Examples:
// Input: arr[][] = [[0,1,1,1], [0,0,1,1], [1,1,1,1], [0,0,0,0]]
// Output: 2
// Explanation: Row 2 contains the most number of 1s (4 1s). Hence, the output is 2.

// User function template for C++
class Solution {
  public:
```

```
   // Binary search to find the first index where value >= x
    int lowerBound(vector<int> &arr, int n, int x) {
        int low = 0, high = n - 1;
        int ans = n;  // Default if x not found

        while (low <= high) {
            int mid = (low + high) / 2;
            if (arr[mid] >= x) {
                ans = mid;       // Possible answer
                high = mid - 1;  // Look for smaller index
            } else {
                low = mid + 1;   // Search right half
            }
        }
        return ans;
    }

    // Find row with max number of 1s
    int rowWithMax1s(vector<vector<int>> &matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        int cnt_max = 0;
        int index = -1;

        for (int i = 0; i < n; i++) {
            int cnt_ones = m - lowerBound(matrix[i], m, 1);  // 1s = total - index of first 1
            if (cnt_ones > cnt_max) {
                cnt_max = cnt_ones;
                index = i;
            }
        }
        return index;
    }
};
```

5.2 LC74:Search in sorted 2D matrix

```
func searchMatrix(matrix [][]int, target int) bool {
    n := len(matrix)
    m := len(matrix[0])
    l := 0
    r := n*m - 1

    for l <= r {
        mid := (l + r) / 2
        // map 1D index back to 2D
        val := matrix[mid/m][mid%m]

        if val == target {
            return true
        } else if val < target {
            l = mid + 1
        } else {
            r = mid - 1
        }
    }
    return false
}
```

5.3 LC240: Search in sorted matrix II

```
// https://leetcode.com/problems/search-a-2d-matrix-ii/
// Write an efficient algorithm that searches for a value target in an m x n integer matrix matrix. This matrix has the f
ollowing properties:
// Integers in each row are sorted in ascending from left to right.
// Integers in each column are sorted in ascending from top to bottom.
func searchMatrix(matrix [][]int, target int) bool {
    i := 0
    j := len(matrix[0]) - 1
    for i < len(matrix) && j >=0 {
        if matrix[i][j] == target {
            return true
        } else if matrix[i][j] < target {
            i++
        }else{
```

```
            j--
        }
    }
    return false
}
```

5.4 LC1901: Find Peak Element II

```
// https://leetcode.com/problems/find-a-peak-element-ii/
// A peak element in a 2D grid is an element that is strictly greater than all of its adjacent neighbors to the left, rig
ht, top, and bottom.
// Given a 0-indexed m x n matrix mat where no two adjacent cells are equal, find any peak element mat[i][j] and return t
he length 2 array [i,j].
// You may assume that the entire matrix is surrounded by an outer perimeter with the value -1 in each cell.
// You must write an algorithm that runs in O(m log(n)) or O(n log(m)) time.
class Solution {
public:
int maxElement(vector<vector<int>>& arr, int col) {
          int n = arr.size();
          int max_val = INT_MIN;
          int index = -1;
          for (int i = 0; i < n; i++) {
              if (arr[i][col] > max_val) {
                  max_val = arr[i][col];
                  index = i;
              }
          }
          return index;
      }
vector<int> findPeakGrid(vector<vector<int>>& arr) {
          int n = arr.size();
          int m = arr[0].size();

          int low = 0;
          int high = m - 1;

          while (low <= high) {
              int mid = (low + high) / 2;

              int row = maxElement(arr, mid);

              int left = mid - 1 >= 0 ? arr[row][mid - 1] : INT_MIN;
              int right = mid + 1 < m ? arr[row][mid + 1] : INT_MIN;

              if (arr[row][mid] > left && arr[row][mid] > right) {
                  return {row, mid};
              }
              else if (left > arr[row][mid]) {
                  high = mid - 1;
              }
              else {
                  low = mid + 1;
              }
          }

          return {-1, -1};
      }
  };
```

5.5 Matrix Median

```
// https://www.naukri.com/code360/problems/median-of-a-row-wise-sorted-matrix_1115473
// Problem statement
// You are given a row-wise sorted matrix 'mat' of size m x n where 'm' and 'n' are the numbers of rows and columns of th
e matrix, respectively.
// Your task is to find and return the median of the matrix.
// Note:
// 'm' and 'n' will always be odd.
// Example:
// Input: 'n' = 5, 'm' = 5
// 'mat' =
// [     [ 1, 5, 7, 9, 11 ],
//       [ 2, 3, 4, 8, 9 ],
//       [ 4, 11, 14, 19, 20 ],
//       [ 6, 10, 22, 99, 100 ],
//       [ 7, 15, 17, 24, 28 ]   ]
```

```
// Output: 10
// Explanation: If we arrange the elements of the matrix in the sorted order in an array, they will be like this-
// 1 2 3 4 4 5 6 7 7 8 9 9 10 11 11 14 15 17 19 20 22 24 28 99 100
// So the median is 10, which is at index 12, which is midway as the total elements are 25, so the 12th index is exactly
midway. Therefore, the answer will be 10.
int count(int target, vector<int> arr){
    int l = -1;
    int r = arr.size();
    while(r > l + 1){
        int mid = l + (r - l)/2;
        if(arr[mid] <= target){
            l = mid;
        }else{
            r = mid;
        }
    }
    return l + 1;
}

int median(vector<vector<int>> &matrix, int m, int n) {
    int tr = 1 + (n * m)/2;

    int l = -1;
    int r = 1e9 + 1;

    while(r > l + 1){
        int mid = l + (r - l)/2;
        int cnt = 0;

        for(int i = 0; i < m; i++){
            cnt += count(mid, matrix[i]);
        }

        if(cnt < tr){
            l = mid;
        }else{
            r = mid;
        }
    }
    return r;
}
```